



LAB REPORT- 02

Department of Computer Science & Engineering (CSE)

Faculty of Science and Information Technology (FSIT)

Summer 2026

Course Title : Computer Graphics Lab
Course Code : CSE412
Course Credit : 1.5
Date of Submission : **June 23 , 2026**
Topic Covered : Design and Implementation of a
2D House Scene Using the DDA Line Drawing Algorithm

Submitted By

Shafin Ahmed

232-15-184

65_M1

Department of Computer Science & Engineering

Submitted To

Husne Mubarak

Lecturer

Department of Computer Science & Engineering

Faculty of FSIT

Comment By Course Teacher:

--

Lab Report 02

Experiment No.: 02

Experiment Title: Design and Implementation of a 2D House Scene Using the DDA Line Drawing Algorithm

Introduction

Overview

In this experiment, a two-dimensional (2D) vector graphics scene displaying two architecturally detailed houses situated side-by-side was successfully engineered and executed. The structural outlines of these objects are generated at a fundamental raster level using a custom implementation of the **Digital Differential Analyzer (DDA) Line Drawing Algorithm**. Instead of offloading rendering computations to built-in high-level line primitives, every horizontal, vertical, and diagonal boundary vector in the scene is calculated incrementally and plotted point-by-point.

The system was configured inside the Code::Blocks integrated development environment (IDE) using the C programming language, linked dynamically with OpenGL runtime contexts and the OpenGL Utility Toolkit (GLUT). To satisfy exact design criteria, the viewing matrix is locked within a clean orthographic grid mapping from **0 to 100** across both dimensions, projected onto a square display container specified exactly at **700 × 700 pixels**. Furthermore, a helper abstraction layer handles the mathematical conversion of standard integer RGB values (0–255) into normalized floating-point components.

The executed system renders two vibrant, symmetrically aligned, structurally proportioned 2D houses side-by-side on a dark background, validating the precision of fractional line step accumulations.

Contents

The software infrastructure is modularized into dedicated structural layers to handle specific mathematical processes, layout frameworks, and OpenGL graphics contexts:

Custom DDA Core Function (drawLineDDA)

Accepts dual endpoint pairs (x_1, y_1) and (x_2, y_2) as float parameters. It evaluates coordinate spans (Δx and Δy), identifies the dominant driving axis via absolute magnitude checks (fabs), allocates the loop iteration limit via a floating-point steps variable, accumulates fractional pixel offsets step-by-step, and pipes bounded matrix entries directly via glVertex2f() wrapped in a GL_POINTS primitive.

Normalized RGB Vector Mapper (setColorRGB)

Translates standard integer-based color formats to floating-point values using the conversion formula:

$$\text{Normalized Intensity} = \text{Integer RGB Component Value} / 255.0$$

Symmetric Geometric Assembly (drawUniqueHouse)

A single parameter-driven blueprint function that uses a translation horizontal scale (xOffset) to draw multiple unique structural units from a single set of vertices without layout overlaps.

Component Configurations

Main Structural Boundaries (Yellow)

A four-vector perimeter forming the load-bearing enclosure.

Extended Roof Slopes (Green)

A triangular canopy designed with extra roof overhang spacing beyond the vertical walls for added visual appeal.

Paneled Entry Portals (Brown)

A multi-line structural door configuration utilizing an extra vertical split down the middle to simulate a double-door panel layout.

Window Ventilation Units (Blue)

Twin windows flanking each door, containing internal cross-bars (window grids) to showcase precise horizontal and vertical line intersections.

Exhaust Chimneys (White)

An offset rectangular assembly aligned with the roof's angle to provide structural variety.

Ground Baseline (Green)

A continuous baseline stretching from $X = 0$ to $X = 100$ at a raised altitude of $Y = 15$ to keep the structures perfectly framed inside the display.

Environment / Platform

Component	Description
Programming Language	C
IDE	Code::Blocks
Graphics Library	OpenGL
Utility Toolkit	GLUT
Operating System	Windows
Algorithm Used	Digital Differential Analyzer (DDA)
Coordinate System	$X = 0-100$, $Y = 0-100$
Window Size	700×700 pixels

Project Information

- **Project Title:** TwoHouse_DDA
- **Project Folder:** LabReport02

- **Project File:** TwoHouse_DDA.cbp

Graph (Desmos)

Before implementing the project in OpenGL, the geometric boundaries and structural lines were mapped out using absolute coordinates within a 100×100 bounding envelope using the Desmos Graphing Calculator. The graph helped determine the coordinates of the walls, roof slopes, door, windows, and chimney accurately before writing the program. It also allowed verification of the alignment, proportions, and placement of both houses within the coordinate range of 0 to 100.

Designing the graph beforehand reduced coordinate tracking errors, eliminated alignment discrepancies, and made the OpenGL matrix implementation highly organized and efficient.

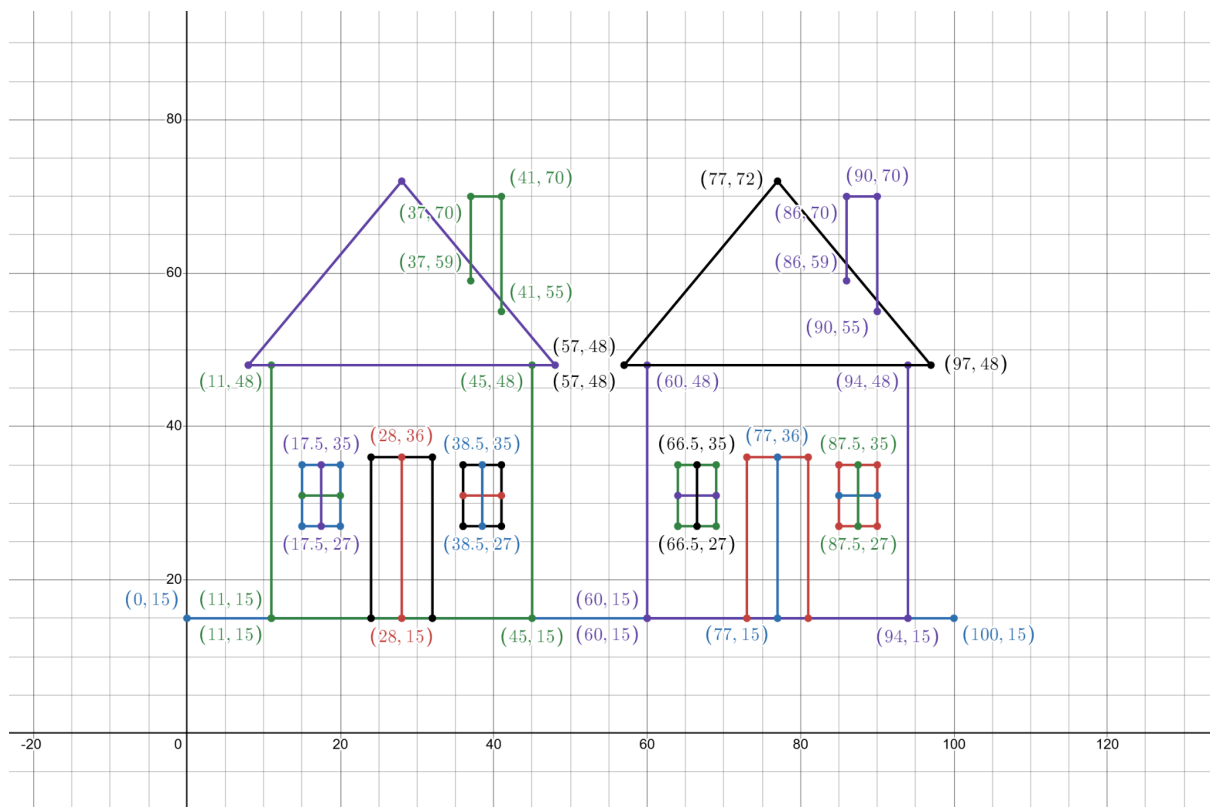


Figure 1: Desmos Graph Layout of the Two Houses

Task Description

The core objective of this laboratory experiment was to design and display two structurally detailed 2D houses side-by-side inside a strictly bounded coordinate matrix without using pre-defined line functions.

The following requirements were successfully completed:

- Designed two highly detailed houses side-by-side on a single coordinate plane.
 - Used exclusively the low-level DDA algorithm for drawing all straight boundary vectors.
 - Changed the coordinate projection matrix mapping to range uniformly from 0 to 100.
 - Initialized the rendering view window container size exactly to 700 × 700 pixels.
 - Converted conventional 8-bit integer RGB color vectors (0–255 format) into OpenGL compatible float parameters.
 - Utilized five distinct structural component colors to satisfy lab constraints.
 - Created a clean and visually distinct graphical output using an aesthetic Brazil FIFA-inspired color theme.
-

Source Code

```
// Shafin Ahmed (232-15-184) (65-M1)
//2D House using DDA
#include <GL/gl.h>
#include <GL/glut.h>
#include <stdio.h>
#include <math.h>

// DDA Line Drawing Algorithm Function
void drawLineDDA(float x1, float y1, float x2, float y2) {
    float dx = x2 - x1;
    float dy = y2 - y1;
    float steps;

    if (fabs(dx) > fabs(dy)) {
        steps = fabs(dx);
    } else {
        steps = fabs(dy);
    }

    float xIncrement = dx / steps;
    float yIncrement = dy / steps;

    float x = x1;
    float y = y1;

    glBegin(GL_POINTS);
```

```

for (int i = 0; i <= steps; i++) {
    glVertex2f(round(x), round(y));
    x += xIncrement;
    y += yIncrement;
}
glEnd();
}

// Function to convert RGB color from 0-255 format to OpenGL format
void setColorRGB(int red, int green, int blue) {
    glColor3f(red / 255.0, green / 255.0, blue / 255.0);
}

// Custom Unique House Design Function
void drawUniqueHouse(float xOffset) {
    // 1. Walls / Main Body (Yellow)
    setColorRGB(255, 223, 0);
    drawLineDDA(8 + xOffset, 15, 42 + xOffset, 15); // Base Floor Line
    drawLineDDA(8 + xOffset, 15, 8 + xOffset, 48); // Left Wall
    drawLineDDA(42 + xOffset, 15, 42 + xOffset, 48); // Right Wall
    drawLineDDA(8 + xOffset, 48, 42 + xOffset, 48); // Top Ceiling Line

    // 2. Roof with Realistic Overhang (Green)
    setColorRGB(0, 156, 59);
    drawLineDDA(5 + xOffset, 48, 25 + xOffset, 72); // Left Slope
    drawLineDDA(45 + xOffset, 48, 25 + xOffset, 72); // Right Slope
    drawLineDDA(5 + xOffset, 48, 45 + xOffset, 48); // Extended Roof Bottom

    // 3. Detailed Door (Brown)
    setColorRGB(139, 69, 19);
    drawLineDDA(21 + xOffset, 15, 21 + xOffset, 36); // Left Frame
    drawLineDDA(29 + xOffset, 15, 29 + xOffset, 36); // Right Frame
    drawLineDDA(21 + xOffset, 36, 29 + xOffset, 36); // Top Frame
    drawLineDDA(25 + xOffset, 15, 25 + xOffset, 36); // Unique Center Panel Line

    // 4. Windows with Cross-bars/Grids (Blue)
    setColorRGB(0, 39, 118);
    // --- Left Window ---
    drawLineDDA(12 + xOffset, 35, 17 + xOffset, 35); // Top
    drawLineDDA(12 + xOffset, 27, 17 + xOffset, 27); // Bottom
    drawLineDDA(12 + xOffset, 27, 12 + xOffset, 35); // Left
    drawLineDDA(17 + xOffset, 27, 17 + xOffset, 35); // Right
    drawLineDDA(14.5 + xOffset, 27, 14.5 + xOffset, 35); // Window Cross Vertical
    drawLineDDA(12 + xOffset, 31, 17 + xOffset, 31); // Window Cross Horizontal

    // --- Right Window ---
    drawLineDDA(33 + xOffset, 35, 38 + xOffset, 35); // Top
    drawLineDDA(33 + xOffset, 27, 38 + xOffset, 27); // Bottom
    drawLineDDA(33 + xOffset, 27, 33 + xOffset, 35); // Left
    drawLineDDA(38 + xOffset, 27, 38 + xOffset, 35); // Right
    drawLineDDA(35.5 + xOffset, 27, 35.5 + xOffset, 35); // Window Cross Vertical
    drawLineDDA(33 + xOffset, 31, 38 + xOffset, 31); // Window Cross Horizontal

    // 5. Chimney (White) - Sits perfectly on the roof slope

```

```

    setColorRGB(255, 255, 255);
    drawLineDDA(34 + xOffset, 59, 34 + xOffset, 70); // Left edge
    drawLineDDA(38 + xOffset, 55, 38 + xOffset, 70); // Right edge
    drawLineDDA(34 + xOffset, 70, 38 + xOffset, 70); // Top rim
}

// Display Callback Function
void display() {
    // Elegant dark background for contrast
    glClearColor(0.08f, 0.08f, 0.08f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Continuous Ground Baseline (Green)
    setColorRGB(0, 156, 59);
    drawLineDDA(0, 15, 100, 15);

    // Draw House 1 (Left Side - shifted slightly right for padding)
    drawUniqueHouse(3);

    // Draw House 2 (Right Side - perfectly spaced side-by-side)
    drawUniqueHouse(52);

    glFlush();
}

// Initialization Function
void init() {
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Custom 0-100 grid configuration
    glOrtho(0.0, 100.0, 0.0, 100.0, -1.0, 1.0);

    // Smooth and thick line rendering configuration
    glPointSize(4.0);
    glEnable(GL_POINT_SMOOTH);
    glHint(GL_POINT_SMOOTH_HINT, GL_NICEST);
}

// Main Function
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);

    // Strict 700x700 size setup
    glutInitWindowSize(700, 700);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("2D Houses using DDA Algorithm");

    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

```



```
}
```

Code Description

Header Files

The following header files were included in the program:

- `#include <GL/gl.h>` : Provides core OpenGL graphics rendering pipeline contexts.
- `#include <GL/glut.h>` : Provides windowing operations, rendering frame loops, and display callback utilities.
- `#include <stdio.h>` : Supports standard terminal input/output debugging stream configurations.
- `#include <math.h>` : Supplies standard mathematical calculations such as absolute values (fabs) and value rounding (round).

DDA Line Drawing Function

A separate mathematical processing function was implemented for the Digital Differential Analyzer algorithm. This routine accepts coordinate endpoints, determines coordinate deltas, determines the driving dominant axis length, sets the loop limits via the steps identifier, applies fractional step increments using floating-point calculations, and passes the rounded integer coordinates to screen space using `GL_POINTS` primitives. This function serves as the foundational drawing engine of the entire project.

Display Function

The `display()` callback is responsible for clearing the screen buffers and triggering all real-time rendering components. It sets a dark background clear color, maps the required RGB color vectors sequentially, and updates coordinate configurations dynamically using a parameterized offset (`xOffset`). This enables the rendering of two distinct houses positioned side-by-side on a continuous green horizontal ground line.

House Construction

Each house was constructed out of distinct mathematical boundary paths. The architectural layout divides coordinates into an outer yellow framing body, an overhanging green slope peak, a centered dual-panel entry frame, and balanced left and right blue window ventilation

grids. Internal cross lines are drawn within the windows to show clear line-on-line intersections.

Color Management

The project mapped colors using an abstract conversion interface. Before passing rendering instructions to the GPU, standard 8-bit RGB color codes are converted into appropriate floating-point signals via a simple custom helper method. This step simplifies coordinate file reading and makes selecting color themes much easier.

Coordinate System

The coordinate transformation is handled by setting an orthographic parallel view projection:

```
glOrtho(0.0, 100.0, 0.0, 100.0, -1.0, 1.0);
```

This converts the standard screen space into an easily traceable grid mapping from 0 to 100 along both primary dimensions, making object layout and positioning highly efficient.

Window Initialization

The execution context utilizes GLUT properties initialized inside the main() function block. The frame properties are hardcoded to a 700 × 700 size, a single frame buffering layer with standard RGB mapping is selected, and execution control is transferred to the main window display loop thread via glutMainLoop().

Overall Program Workflow

1. Initialize the GLUT engine context state.
 2. Build the display canvas window framework container.
 3. Configure projection parameters and set the coordinate plane limits to 0–100.
 4. Pass processing control to the display callback loop handler.
 5. Generate structural coordinates sequentially using point-by-point calculation updates.
 6. Refresh internal buffers using glFlush() to project the finalized 2D structural display onto the screen.
-

Output

Output Screenshot

The program successfully initialized a 700×700 dark rendering environment and displayed two colorful 2D houses side-by-side using the Digital Differential Analyzer (DDA) Line Drawing Algorithm. The structural lines look uniform, bold, and highly distinct due to the point-smoothing enhancements and calibrated point-size scaling configured in the initialization block.

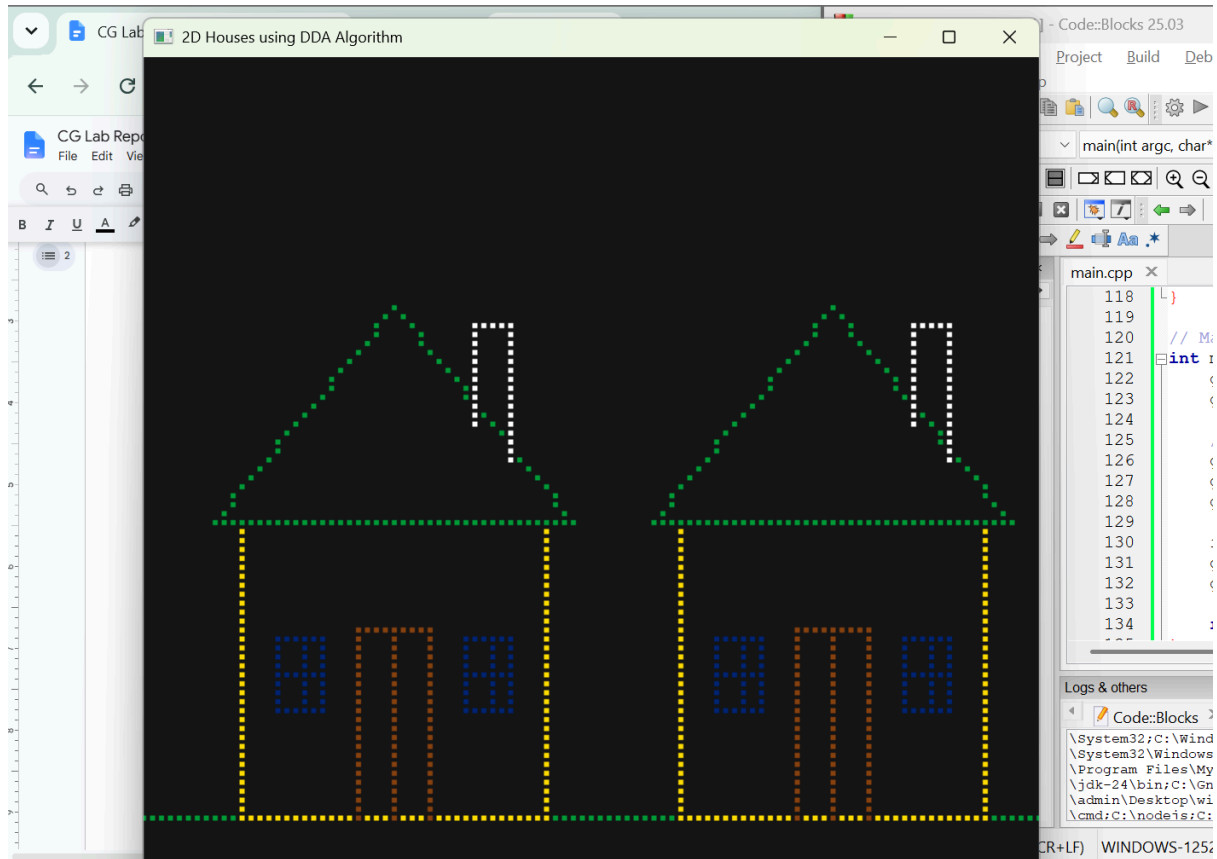


Figure 2: Finished Output of Two 2D Houses Drawn Using DDA Algorithm

Discussion

In this experiment, I implemented the Digital Differential Analyzer (DDA) Line Drawing Algorithm to construct a complete 2D house scene. Instead of using OpenGL's built-in line drawing functions, I calculated every intermediate point manually using the DDA algorithm and plotted those points individually. This allowed me to understand how a line is generated mathematically on the computer screen.

The project consisted of two houses placed side by side. Each house included a roof, walls, windows, a door, and a chimney. Different RGB color values in the 0–255 format were converted into OpenGL's normalized color format before drawing. The coordinate system was modified to 0–100, making the object placement easier and more organized.

Before implementing the project, I designed the structure in Desmos to estimate the coordinates of every line accurately. After confirming the coordinate positions, I transferred those values into the OpenGL program. This approach reduced coordinate errors and made debugging much easier.

Throughout the implementation, I divided the drawing into small reusable functions so that the program remained clean, readable, and easier to modify. The DDA function was reused to draw every edge of both houses.

Challenges Faced

1. Coordinate Apportioning.
2. Horizontal Layout Management.
3. Rasterization Point Separation.
4. Diagonal Vector Continuity.

I solved these problems by first plotting the coordinates in Desmos, checking the output repeatedly, and then adjusting the coordinates inside the OpenGL program. To fix the disconnected point issue, I used:

```
glPointSize(4.0);
```

along with anti-aliasing configurations (`GL_POINT_SMOOTH`) within the runtime initialization phase.

Conclusion

This laboratory experiment provided a comprehensive understanding of the foundational operational logic of the Digital Differential Analyzer (DDA) Line Drawing Algorithm. It successfully demonstrated how programmatic math loops step through spatial locations to form vectors pixel-by-pixel, eliminating reliance on high-level black-box primitive wrappers.

Valuable practical insights were gained regarding orthographic grid calibration, bit-depth color normalization, structural layout reuse via variable offsets, and primitive state configurations. Planning the layout in Desmos before writing the program highlighted the importance of structured design preparation in reducing software debugging time. Ultimately, this experiment strengthened my understanding of computer graphics fundamentals, providing a strong foundation for exploring advanced concepts in fields like Computer-Aided Design (CAD), user interface architecture, and real-time interactive game development loops.

Discussion Questions

1. What is the purpose of the DDA algorithm?

The main purpose of the Digital Differential Analyzer (DDA) algorithm is to calculate the intermediate pixel locations that form a straight line path between two defined endpoints (x_1, y_1) and (x_2, y_2) . It serves as a fundamental scan-conversion technique that samples a line at unit intervals along one coordinate axis while incrementally calculating fractional steps along the other axis.

2. Why does DDA use floating-point arithmetic?

DDA uses floating-point arithmetic because the calculated coordinate increments (Δx / steps or Δy / steps) are often fractional values. Using floating-point tracking variables preserves these subtle fractional values during loop iterations, preventing early truncation errors and ensuring that line plotting remains accurate before rounding to integer pixel coordinates.

3. What happens when the slope is greater than 1?

When the slope $m > 1$, the vertical change (Δy) is larger than the horizontal change (Δx), meaning y is the dominant axis. In this scenario, the algorithm increments the y -coordinate by exactly 1.0 unit per loop step while computing a fractional step increment for the corresponding x -coordinate ($x_{\text{new}} = x_{\text{old}} + 1/m$), preventing gaps in steep lines.

4. What is the role of the variable "steps"?

The variable `steps` determine the absolute number of loop iterations needed to complete a continuous line path. It is set to the maximum absolute distance along either axis, $\max(|\Delta x|, |\Delta y|)$. This ensures that the algorithm increments along the dominant axis by exactly one unit per loop iteration, providing complete and unbroken pixel coverage across the screen.

5. Why are OpenGL points used instead of directly drawing a line?

OpenGL points (`GL_POINTS`) are used to demonstrate the lower-level mechanics of scan conversion. By plotting every point individually, the program directly showcases how the DDA mathematical algorithm generates shapes pixel-by-pixel, rather than hiding these calculations behind OpenGL's high-level line primitives (`GL_LINES`).

6. What are the limitations of DDA?

The limitations of the DDA algorithm include:

- It relies heavily on floating-point arithmetic and fractional accumulations, which can slow down hardware operations compared to pure integer workflows.
- The need to run a rounding function (`round()`) during every single iteration introduces performance overhead.

- Rounding errors can accumulate over long distances, which can cause the drawn line to drift slightly from its true mathematical path.

7. Why is Bresenham's algorithm generally faster than DDA?

Bresenham's algorithm is significantly faster because it operates entirely on integer arithmetic, eliminating the need for floating-point additions or division steps. It relies on a fast decision parameter that uses integer addition, subtraction, and bit-shifting operations to determine the next pixel location, which executes much quicker at the hardware level.